MOAT codebase handover

A run-through for the incoming developer: what the app is for, how it is built, what every file does, the user flows, the lesson plans, and what needs attention first.

Project folder: `moat3new_updated` · React Native 0.86 / Expo SDK 57 / TypeScript

Written 10 October 2026 from a read of every source and content file in the folder.

Checks run while writing: `tsc --noEmit`, content-file statistics, and cross-references between code and lesson JSON.

Contents

1 What MOAT is
2 Running it, and the tech stack
3 Architecture at a glance
4 The key idea: two learning systems
5 User flows (diagrams)
6 Navigation map
7 State, rules and persistence
8 File-by-file reference
9 Content formats
10 Lesson plans
11 Mini-games
12 Payments and the backend hooks
13 Design system and accessibility
14 Known issues and gaps
15 How-to recipes
16 Suggested first week, and glossary

> **If you only read one page, read section 4.** The app contains two separate learning systems (the older *general course* and the newer *project track*) with their own data shapes, screens, scoring and progress stores. Almost every "where does this live?" question is answered by knowing which of the two you are in.

Five things to know before you touch anything

1. **There is no backend and no account system.** All lessons are JSON bundled in `assets/content`; all progress lives in AsyncStorage on the phone under two keys.
2. **It is a port of a SwiftUI app.** Comments such as "LessonView.swift" or "TrackKit_updated.swift" name the original Swift file each piece was translated from. The Swift code is not in this folder.
3. **Files ending in `_updated` are the live ones.** There are no un-suffixed twins inside `src/`; the suffix is historical. The README still uses the old names.
4. **Three files in the project root are stale copies** (`LearnScreen.tsx`, `TrackPathScreen.tsx`, `ui.tsx`). Nothing imports them and they are the only source of type errors. Delete them.
5. **Buying Premium currently unlocks nothing.** The paywall and RevenueCat calls work, but no screen reads `isPremium`. See section 12.

1 What MOAT is

MOAT is a phone app that teaches complete beginners to code in short, tap-only lessons. It borrows the familiar habit-app shape: a path of lessons that climbs up the screen, five quick questions per lesson, a daily streak, a currency ("bolts"), stars for accuracy, and a robot mascot that reacts to your answers and evolves into new forms.

Its distinguishing idea is that **the lessons are written around the learner's own field**. During onboarding the learner picks an area of study and a programming language. A healthcare learner calculates drug doses and reads a pulse monitor; an engineer sorts fruit on a conveyor and applies Ohm's law; a biology researcher builds a DNA sequence report in R. The same programming concepts are taught each time, but the story, the variable names and the pictures change.

Audience	Ages 10 to 70+. Under-13s get a "a parent will need to help" note, a lighter stored daily goal, and hints that are always visible.
Study areas	Healthcare, Biology research, Engineering, Business & finance, Physics & maths, Arts & humanities, General.
Languages	Python, R, SQL, JavaScript, MATLAB, Java. Depth varies a lot by combination (section 4).
Interaction	Tapping only: choose an option, tap code pieces into order, tap the buggy line, reorder steps. The single typed input in the whole app is a number in the loop mini-game.
Rewards	Bolts (points), stars (first-try accuracy on track lessons), day streak, nine badges, robot forms.
Business model	Subscription paywall with two tiers defined (Premium $9.90/month, Expert $19.90/month, 7-day trial) through RevenueCat. Gating is not implemented yet.
Platforms	iOS and Android through Expo. Portrait only, light mode only, phones only (`supportsTablet: false`).
Maturity	Version 1.0.0, pre-release. Runs in Expo Go. No tests, no lint config, no CI, no git history, no store build configuration.

What is finished and what is not

Working end to end

- Splash, two-step onboarding, three-tab shell.
- General course: 3 phases, 12 lessons, for five Python study-area themes; a Phase 1 starter course for R, SQL, JavaScript, Java and MATLAB.
- Biology research track in R: 5 phases, 20 lessons, 5 checkpoints.
- New-style animated lessons: 27 of the 35 track lessons.
- Streaks, bolts, stars, badges, robot evolution, workshop.
- Paywall UI and RevenueCat test-store purchase flow.

Stubbed, partial or missing

- Engineering and Healthcare tracks: only Phase 1 plus one sample lesson from a later phase; no checkpoints.
- Biology phases 4 and 5 still use the older player.
- "Cycle 2" is a teaser card only.
- Premium entitlements do not change behaviour.
- Backend lesson fetch and "Explain more" AI help are switched off; no server exists in this folder.
- Four of six robot forms and two of three robot parts are "coming soon".
- Sign-in, sync, account deletion.

2 Running it, and the tech stack

```
npm install
npx expo install --fix      # pins packages to the Expo SDK your Expo Go supports
npx expo start              # scan the QR code with Expo Go (add --lan --clear on restricted Wi-Fi)
npm run typecheck           # tsc --noEmit
```

`node_modules` is already installed in the folder. The project targets Expo SDK 57; if your Expo Go is on another SDK, run `npx expo install expo@<sdk> --fix` first. To reach the project tracks while testing, choose Biology research, or Engineering + Python, or Healthcare + Python or R in onboarding. To reach the mini-games choose Business & finance + Python.

Concern	Library	Notes
Runtime	Expo ~57, React Native 0.86.3, React 19.2.3	Entry is `index.ts` (`"main"` in package.json), not expo-router.
Language	TypeScript ~6.0, `strict` on	`resolveJsonModule` on, because content is imported as JSON.
Navigation	@react-navigation native-stack 7 + bottom-tabs 7	One root stack, one tab navigator. Typed param lists.
State	React Context only	Two providers. No Redux, Zustand or query library.
Persistence	@react-native-async-storage/async-storage	Two keys, whole-object JSON, written on every change.
Drawing	react-native-svg	Icons, phase pictures, lesson pictures and mini-game art are all drawn in code.
Animation	React Native `Animated`	No Reanimated. Every animation checks Reduce Motion.
Payments	react-native-purchases 9 (RevenueCat)	Runs in "Preview API Mode" inside Expo Go: calls work, purchases are mocked.
Other Expo modules	expo-font, expo-haptics, expo-blur, expo-status-bar	Fonts loaded in `App.tsx`; haptics on answer check; blur behind the paywall.
Icons	@expo/vector-icons (Ionicons)	Replaces SF Symbols from the Swift app.
Unused dependency	@react-native-community/slider	Installed, but onboarding draws its own slider with `PanResponder`.

> **Type-check status (run 10 Oct 2026):** `tsc --noEmit` reports 19 errors, and all 19 are in the three stale root-level files. Everything under `src/` and `App.tsx` compiles cleanly under `strict`. The README's "Not verified, expect type errors" section is out of date on this point.

Size of the codebase

Area	Files	Lines	Comment
`src/components`	7	about 2,870	Three files (lessonKitV2, trackKit, picturesV2) hold most of the visual code.
`src/screens` (incl. `games/`, `track/`)	15	about 2,840	
`src/hooks`, `services`, `storage`, `utils`, `types`, `navigation`	18	about 1,220	All of the business rules live in under 700 lines here.
`assets/content`	19 JSON	about 800 KB	The lesson plans. This is where most product value sits.

`assets/images`	92 images		61 registered in `utils/images.ts`, 31 fruit sprites required directly by `picturesV2.tsx`.

3 Architecture at a glance

The app is a conventional layered React Native app. Dependencies point downward only: screens use components and hooks; hooks use services and storage; services read static content. Nothing below the hooks layer knows about React Navigation.

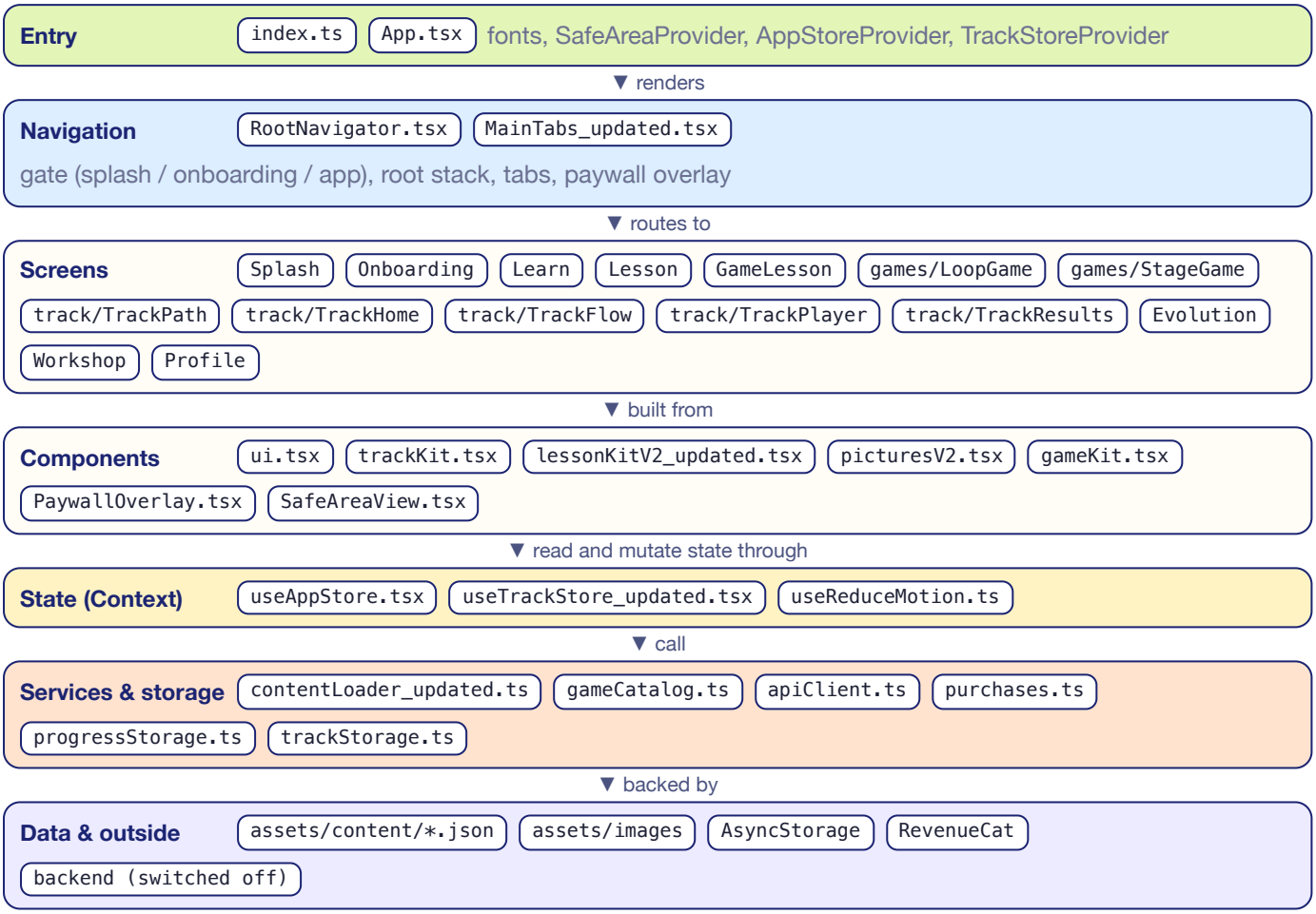


Figure 1. Layers and the files in each. Shared foundations used by every layer: `types/models.ts` (all data shapes) and `utils/theme.ts` (colours and type styles).

Conventions you will see everywhere

- **Inline styles, no StyleSheet files.** Layout is written as style objects in JSX, using tokens from `utils/theme.ts` (`C` colours, `BORDER`, `heading()`, `body()`, `code()`, `monoBold()`).
- **Stores keep a ref beside state.** Both providers hold `progressRef` so that callbacks always read the latest value, then call one `setProgress` that updates the ref, React state and AsyncStorage together.
- **Static registries for assets.** Metro needs literal `require()` calls, so every image is listed in `utils/images.ts` and every content file in `utils/content_updated.ts`. A file that is not registered does not exist as far as the app is concerned.
- **Content is data, screens are generic.** No screen contains lesson text. Adding lessons is nearly always a JSON change.
- **Ids are the contract.** Progress is stored as lists of lesson, checkpoint and question ids. Renaming an id in JSON silently discards that progress for existing users.
- **Accessibility is first-class.** Pressables carry roles, labels and states; touch targets are at least 44 pt; body text is Atkinson Hyperlegible.
- **Hook-dependency lint rules are suppressed in places** with `// eslint-disable-line react-hooks/exhaustive-deps`, although no ESLint config is present.

4 The key idea: two learning systems

The Learn tab shows one of two completely different experiences. `MainTabs` decides with a single call: `hasTrack(studyArea, language)`.

	General course (older)	Project track (newer)
Idea	Short themed drills on one concept at a time.	Each phase builds one real project (a script) over four lessons, then tests it with a harder checkpoint.
Home screen	`screens/LearnScreen.tsx`	`track/TrackPathScreen`, then `track/TrackHomeScreen` (the phase page)
Player	`LessonScreen`, or `GameLessonScreen` for mini-games	`track/TrackFlowScreen`, which hosts `LessonPlayerV2` (new style) or `TrackPlayer` (older style and all checkpoints)
Content files	`lessons*.json`	`track_*.json` for structure, plus `track_bio_r_lessons_v2.json` and `track_lessons_v2_more.json` for new-style lesson bodies
Types	`Course, Phase, Lesson, Question`	`Track, TrackPhase, TrackLesson, TrackCheckpoint, TrackQuestion`; `V2Lesson, V2Question`
Question kinds	choice, build, judge, bug, match	predict, fill, bug, build; checkpoints add order (styles: plan, code, predict, debug, build)
Wrong answer	Shows the answer; the question comes back once at the end of the lesson.	"Try again" on the same question until it is right. Only first-try answers earn a star.
Bolts	5 per question right first time, +10 for finishing. Maximum 35. Replays pay again.	10 per question right first time, paid once ever per question; +30 for a perfect first attempt. Maximum 80.
Unlocking	Strictly linear: a lesson opens when every earlier lesson is complete.	Next lesson: previous one finished (any stars). Next phase: 5/5 stars on every lesson and on the checkpoint.
Progress store	`useAppStore`: `progress.completedLessons` (key `moat.progress.v1`)	`useTrackStore`: `finished, bestStars, paid, secondsByDay` (key `moat.track.v1`)
Robot evolution	Yes, when a phase with `unlocksForm` completes.	No. Track learners never evolve the robot today.
Time tracking	None.	Seconds per day, shown as "minutes today".

Shared between both: the learner profile, total bolts and the streak (all in `useAppStore`), the robot, the stat chips at the top of the Learn tab, the Workshop and Profile tabs.

Which learner gets what

Resolved by `services/contentLoader_updated.ts`. A language file always wins over a study-area file; a track wins over both on the Learn tab.

Study area	Python	R	SQL, JavaScript, MATLAB, Java
Biology research	Not selectable. Onboarding forces R and disables the other languages.	**Track** `track_bio_r_updated` 5 phases, 20 lessons, 5 checkpoints	Not selectable.
Engineering	**Track** `track_eng_py` Phase 1 (4 lessons) + Phase 3 (1 lesson)	General: `lessons_r` (Phase 1 only)	General: `lessons_<language>` (Phase 1 only)
Healthcare	**Track** `track_health_py` Phase 1 (4) + Phase 3 (1)	**Track** `track_health_r` Phase 1 (4) + Phase 4 (1)	General: language file, Phase 1 only

Business & finance	General: `lessons_business`, 3 phases, **plus the two mini-games**	General: `lessons_r`	General: language file
Physics & maths	General: `lessons_physics_and_maths`	General: `lessons_r`	General: language file
Arts & humanities	General: `lessons_arts_and_humanities`	General: `lessons_r`	General: language file
General	General: `lessons_general`	General: `lessons_r`	General: language file

Consequences worth knowing.

- `lessons_engineering.json` and `lessons_life_sciences.json` are loaded into the store for Engineering/Healthcare + Python learners but never displayed, because those learners see a track instead.
- `lessons_computing.json` is unreachable: "Computing" is no longer a study area (`canonicalArea` maps it to "General"). `lessons.json` is only a last-resort fallback.
- The health and engineering mini-games (plant, GC content, wheel, duty cycle) can no longer be reached from the UI for the same reason. Only the business pair (fund, conversion) is reachable.
- The language starter files do not receive mini-games; `loadBundled` returns before `addingGames` for them.

5 User flows

5.1 Launch, onboarding and the tab shell

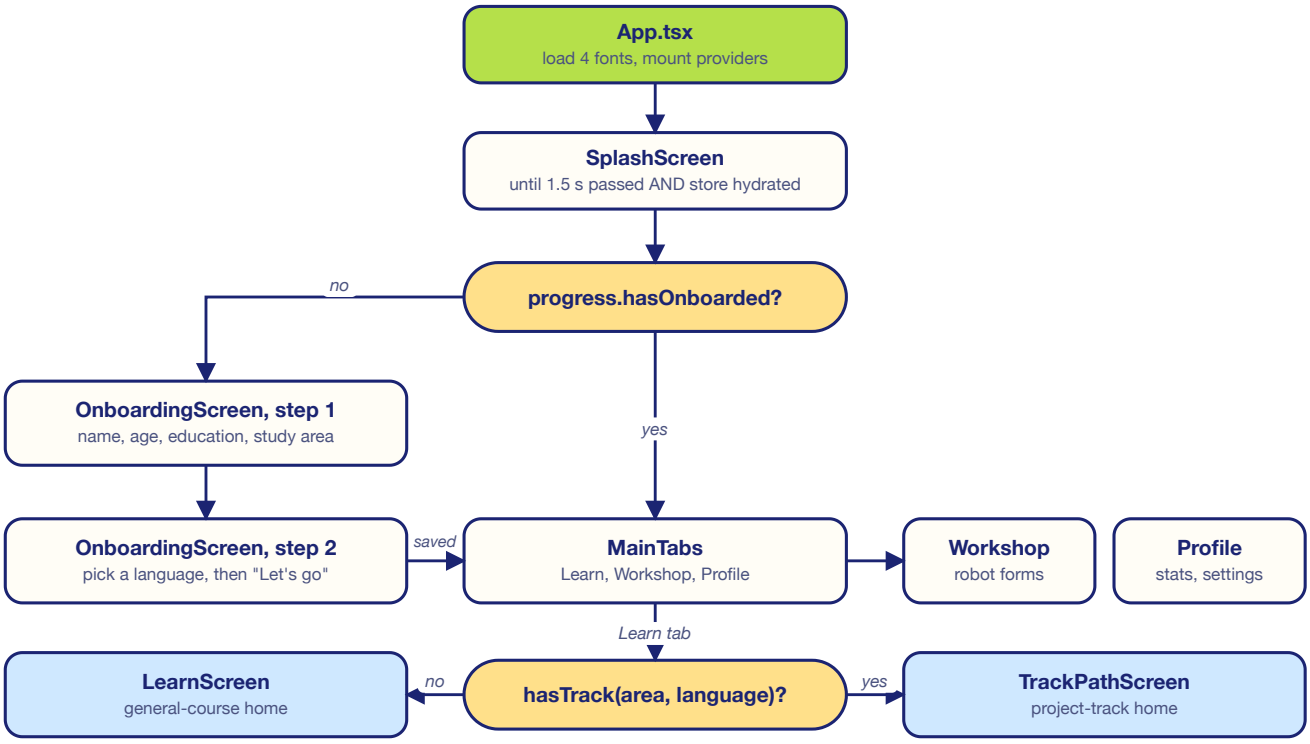


Figure 2. `RootNavigator` is a three-way gate, not three routes: splash and onboarding are rendered *instead of* the navigator. Yellow pills are decisions. "Edit my answers" on Profile sets `hasOnboarded` back to false, which re-enters onboarding with the fields prefilled and all progress kept.

- **Hydration.** `AppStoreProvider` loads saved progress, then the matching bundled course, then sets `hydrated`. The splash stays up until both the 1.5-second timer and hydration are done, so the learner never sees a flash of default content.
- **Onboarding step 1** collects an optional first name (30 characters), age on a custom 10 to 70+ slider, highest education from a modal list, and one of seven study areas.
- **Onboarding step 2** lists the six languages. Languages that have a project track for the chosen area are sorted first and captioned "project lessons". Biology research locks the choice to R.
- **`finishOnboarding()`** saves the answers, sets `hasOnboarded`, sets the stored daily lesson goal (2 if under 13, otherwise 3) and loads the course.

5.2 General course: one lesson

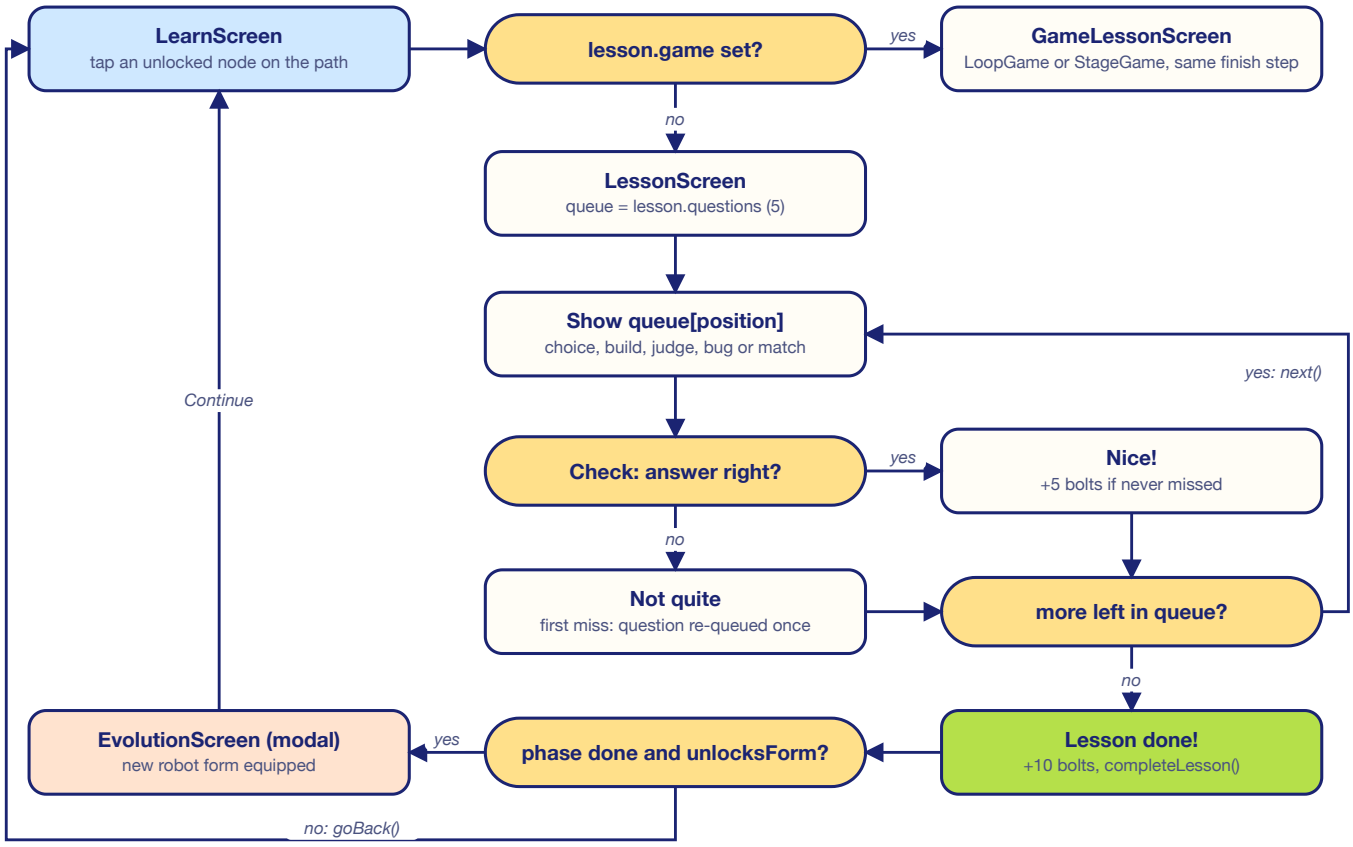


Figure 3. The loop inside `LessonScreen`. A missed question is appended to the queue once; a second miss marks it finished without bolts. The evolution modal is deliberately opened by `LearnScreen` on focus (via `pendingEvolution`) so it appears only after the lesson modal has fully closed.

- The path is drawn bottom-up (rows are reversed) and auto-scrolls to keep the current lesson centred. The robot stands beside the current node.
- Tapping a node in a phase the learner has not reached opens the Premium paywall ("skip with Premium"). Nothing is actually skipped after purchase yet.
- Quitting mid-lesson shows a custom `QuitSheet` and awards nothing.
- Under-13 learners always see the hint; others tap "Need a hint?".

5.3 Project track: path, phase page, lesson, checkpoint

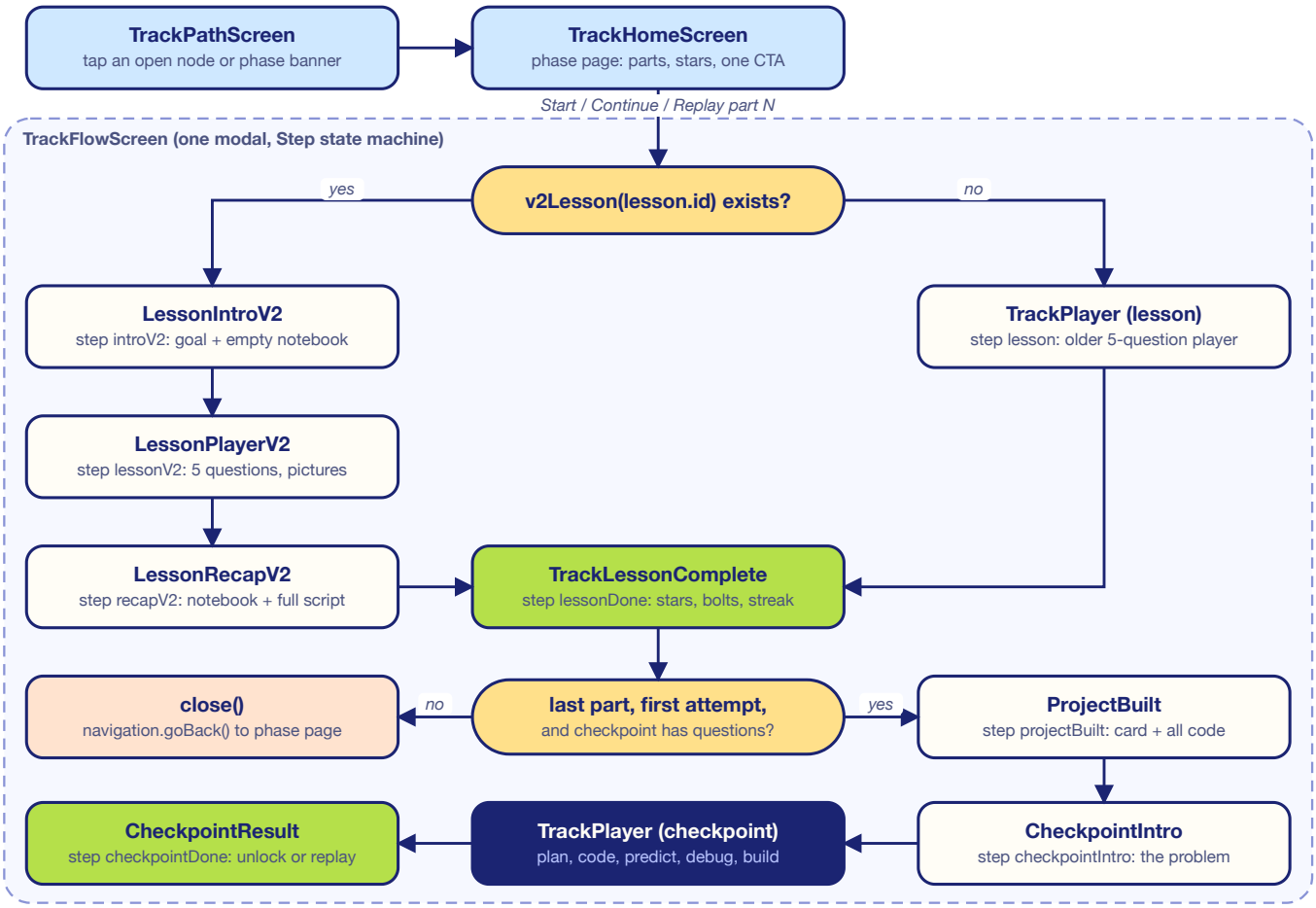


Figure 4. Everything inside the dashed frame is one screen, `TrackFlowScreen`, switching on a `Step` value. The phase page can also enter the flow directly at `CheckpointIntro` ("Take the checkpoint", route param `lessonId: null`). "Review the ones I missed" on the completion screen and "Replay the checkpoint" on the result screen restart the relevant player with a new React `key`.

- **TrackPathScreen** mirrors the general path, but nodes are lessons and checkpoints of every phase, finished nodes show their star count, and tapping opens the phase page rather than a lesson.
- **TrackHomeScreen** shows one phase card with its picture, each part's state and stars, a "Replay" chip on any part below 5 stars, and a single call-to-action chosen by `ctaFor()`: Start part 1, Continue part N, Replay part N, or Take the checkpoint.
- **New-style lesson** (V2): goal screen with a message from a named person and an empty "lab notebook", five questions with a moving picture and a growing script, then a recap with the notebook filled in and the full script.
- **On finish**, the player calls `track.record()` (stars, paid questions, bonus), `app.creditTrack()` (bolts, streak) and `track.addTime()` (minutes).
- **Checkpoint**: five harder questions on a new problem in "navy exam mode", one of each style. 5/5 here plus 5/5 on all four lessons unlocks the next phase.

6 Navigation map

```
RootNavigator                              src/navigation/RootNavigator.tsx
├─ <SplashScreen/>                         while !splashDone || !store.hydrated
├─ <OnboardingScreen/>                     while !store.progress.hasOnboarded
└─ NavigationContainer → native stack      headerShown: false everywhere
   ├─ Main         MainTabs                 bottom tabs
   │  ├─ Learn     TrackPathScreen if hasTrack(area, language), else LearnScreen
   │  ├─ Workshop  WorkshopScreen
   │  └─ Profile   ProfileScreen
   ├─ Lesson       LessonScreen             fullScreenModal, no swipe-back   { lessonId }
   ├─ GameLesson   GameLessonScreen         fullScreenModal, no swipe-back   { lessonId }
   ├─ TrackPhase   TrackHomeScreen          normal push                      { phase }
   ├─ TrackFlow    TrackFlowScreen          fullScreenModal, no swipe-back   { phase, lessonId | null
}
   └─ Evolution    EvolutionScreen          modal sheet                      { form }
+ <PaywallOverlay/>                        absolute overlay above the navigator while
store.showPaywall
```

Route names and params are typed in `RootStackParamList` (exported from `RootNavigator.tsx`) and `TabParamList` (`MainTabs_updated.tsx`). Screens receive only ids and look the objects up in the stores, so nothing large is passed through navigation.

The paywall is not a route. It is a sibling view rendered over the whole navigator, opened with `store.openPaywall(tier)` and closed with `store.closePaywall()`. Lesson and checkpoint players use full-screen modals with gestures disabled so a swipe cannot bypass the quit confirmation.

7 State, rules and persistence

7.1 `useAppStore`: profile, general course, bolts, streak, paywall

Member	Kind	What it does
`hydrated`	state	True once saved progress and the bundled course are loaded. Gates the splash.
`progress`	state, persisted	The `UserProgress` object (below). Every change goes through the internal `setProgress`, which also saves.
`course`	state	The general `Course` for this learner, from `loadBundled(studyArea, language)`. Reloaded on onboarding and reset.
`orderedLessons`, `currentLesson`	derived	All lessons flattened in order; the first one not completed.
`isCompleted, isUnlocked, isPhaseStarted, completedCount`	selectors	Linear unlock rule: a lesson is unlocked when every earlier lesson id is in `completedLessons`. A phase is "started" when its first lesson is unlocked.
`displayStreak`	derived	The stored streak, but 0 if the last active day is neither today nor yesterday.
`lessonsToday`	derived	Lessons finished today. Computed but not shown anywhere yet.
`finishOnboarding(name, age, education, studyArea, language)`	action	Saves answers, sets `hasOnboarded`, sets the daily lesson goal, reloads the course.
`completeLesson(lesson, bolts) → RobotForm | null`	action	Bumps the day and streak, adds bolts (always), records the lesson id (first time only), and if that completes a phase with `unlocksForm`, unlocks and equips the form and returns it.
`creditTrack(bolts)`	action	The track equivalent: bumps the day and streak and adds bolts. Does not touch `completedLessons` or forms.
`equip(form), setName, editAnswers, resetProgress`	actions	Workshop equip (only owned forms); profile name; re-enter onboarding; wipe to defaults.
`pendingEvolution, setPendingEvolution, clearPendingEvolution`	state	Hand-off so the Learn screen can open the Evolution modal after the lesson modal closes.
`showPaywall, paywallTier, openPaywall, closePaywall`	state	Paywall visibility and which tier's card to show.
`isPremium, currentOffering, loadOffering, purchase, restorePurchases`	RevenueCat	Entitlement flag and offering, kept in sync by a customer-info listener. **No screen reads `isPremium`; nothing calls `restorePurchases`.**

Streak and day logic (`bumpDay`, private to the provider)

```
today = DayKey.today()                    // "2026-10-10", device local time
if (lastActiveDay !== today) {
  streak = (lastActiveDay === yesterday) ? streak + 1 : 1
  lastActiveDay = today
}
if (todayKey !== today) { todayKey = today; todayLessons = 0 }
todayLessons += 1
```

It runs whenever a general lesson, mini-game, track lesson or checkpoint is finished. Quitting part-way never counts.

7.2 `useTrackStore`: stars, unlocks, payouts, minutes

Member	Kind	What it does
`track`	derived	`loadTrack(studyArea, language)`, or null when this learner has no track.
`progress`	state, persisted	The `TrackProgress` object (below).
`isFinished(id), stars(id)`	selectors	Whether a lesson or checkpoint id has ever been completed; its best first-try count.
`phaseComplete(phase)`	selector	Every lesson has stars ≥ its question count and the checkpoint has stars ≥ its question count. A checkpoint with no questions passes automatically.
`phaseUnlocked(phase)`	selector	First phase in the array, or the previous phase *in the array* is complete. Position-based, not number-based.
`lessonUnlocked, checkpointUnlocked`	selectors	Phase unlocked and previous lesson finished; phase unlocked and all lessons finished.
`currentPhase, cycleOneComplete, nextLesson(phase)`	derived	The furthest unlocked phase; all phases complete; first unfinished lesson of a phase.
`pays(question)`	selector	True if this question id has never paid bolts. Drives the "+10" animation.
`record(itemID, questions, firstTry) → RunResult`	action	The scoring function. See below.
`addTime(seconds), minutesToday`	action, derived	Adds to today's bucket in `secondsByDay`. `DAILY_GOAL_MINUTES` is 10.
`reset()`	action	Clears track progress. Profile's reset calls this and `resetProgress()` together.

What `record()` does when a track lesson or checkpoint ends

```
firstAttempt = itemID not yet in finished
stars        = number of questions answered right on the first try this run
earned       = 10 × (first-try questions that have never paid before)
bonus        = 30 if firstAttempt and stars === questions.length, else 0
save: finished += itemID;  bestStars[itemID] = max(old, stars);  paid += first-try ids
return { itemID, stars, total, earned, bonus, firstAttempt, missedIndexes }
```

So a replay can raise your stars and pays only for questions you originally missed; the +30 bonus is available on the very first run only.

7.3 What is saved on the phone

Key `moat.progress.v1` (`UserProgress`)

```
{
  "hasOnboarded": true,
  "name": "Sam", "age": 18,
  "education": "Secondary school",
  "studyArea": "Healthcare",
  "language": "Python",
  "completedLessons": ["p1l1", "p1l2"],
  "bolts": 70, "streak": 3,
  "lastActiveDay": "2026-10-10",
  "todayKey": "2026-10-10",
  "todayLessons": 2,
  "dailyGoalLessons": 3,
  "unlockedForms": ["blob"],
  "currentForm": "blob"
}
```

Key `moat.track.v1` (`TrackProgress`)

```
{
  "finished":  ["bio_p1_l1", "bio_p1_cp"],
  "bestStars": { "bio_p1_l1": 4,
                 "bio_p1_cp": 5 },
  "paid":      ["bio_p1_l1_v2_q1", "..."],
  "secondsByDay": { "2026-10-10": 412 }
}
```

`ProgressStorage.decode` reads field by field with defaults, so adding a field later never wipes saved data. It also passes the study area through `canonicalArea()` to migrate old names. Saving is best-effort and never throws.

Both objects are global, not per track or per course. General lesson ids such as `p1l1` are shared across all study-area files, so switching area keeps general progress; track ids are unique per track.

8 File-by-file reference

8.1 Project root

File	Purpose	Details
`index.ts`	Entry point.	`registerRootComponent(App)`.
`App.tsx`	Root component.	Loads four fonts with `useFonts` and shows a cream placeholder until ready. Provider order is `SafeAreaProvider → AppStoreProvider → TrackStoreProvider → RootNavigator`; the order matters because the track provider calls `useAppStore()`.
`app.json`	Expo config.	Name, slug, portrait, light UI, no tablet, `expo-font` plugin. No bundle identifier, icon, splash or EAS settings yet.
`package.json`, `tsconfig.json`	Scripts and compiler settings.	Scripts: `start, android, ios, typecheck`. TS extends `expo/tsconfig.base` with `strict` and `resolveJsonModule`.
`README.md`	Migration notes.	Swift-to-RN mapping, Expo Go notes, feature parity list. File names in it omit the `_updated` suffix and its "Not verified" section is outdated.
`TESTING_playwright-cli.md`	Manual test notes.	How the new-style lesson screens were checked against the design canvas with playwright-cli, and what to look for on each lesson (picture idles, acts out the right answer, shakes on wrong, Reduce Motion holds still).
`LearnScreen.tsx`, `TrackPathScreen.tsx`, `ui.tsx`	**Stale copies. Delete.**	Older versions of the files in `src/`. Not imported by anything; their relative imports do not resolve from the root, which causes all 19 type errors.
`.expo/`, `.gitignore`	Local Expo state; ignore rules.	There is a `.gitignore` but the folder is not a git repository yet.

8.2 `src/types` and `src/utils`

File	Purpose	Key exports
`types/models.ts`	Every data shape in the app, in one file.	Robot: `RobotForm, ROBOT_FORMS, ROBOT_NAMES, RobotMood, moodImage(form, mood), isInVersionOne`. General course: `Question, Lesson, Phase, Course`. Profile: `UserProgress, defaultProgress()`. Track: `TrackQuestion, TrackLesson, TrackCheckpoint, TrackPhase, TrackCycle2, Track, TrackProgress, RunResult, isPerfect`. Helpers `correctOne, correctMany, labelOf` read a track question's answer and option labels.
`utils/theme.ts`	Design tokens.	`C` (about 40 named colours), `BORDER` = 2.5, `FONTS`, and text-style factories `heading(size), body(size, bold), code(size), monoBold(size), wordmark(size)`.
`utils/images.ts`	Generated image registry.	`IMAGES`: name → `require()`. Robot art is named `<form>_<mood>_updated`.
`utils/content_updated.ts`	Generated content registry.	`CONTENT`: file name → parsed JSON for 13 course files and 4 track files. The two V2 lesson files are *not* here; `lessonKitV2` imports them directly.

`utils/studyAreas.ts`	The list of areas and a name migration.	`STUDY_AREAS` (7), `canonicalArea(name)` maps old names ("Health sciences", "Life sciences", "Business", "Computing") to current ones.
`utils/dayKey.ts`	Day labels for streaks.	`DayKey.today(), DayKey.yesterday()` as local-time `YYYY-MM-DD`.
`utils/codeJoin.ts`	Token joining.	`codeJoin(pieces)` turns `["print","("," ","\"Hi\"",")"]` into `print("Hi")` with sensible spacing; used to compare "build" answers. `money(n)` formats dollars.

8.3 `src/services` and `src/storage`

File	Purpose	Key functions
`services/contentLoader_updated.ts`	Decides which content a learner gets. The routing table in section 4 is this file.	`loadBundled(area, language)`: language file if one exists, else the area file passed through `addingGames`, else `lessons.json`. `fileName(area)` and `languageFileName(language)` do the name mapping. `trackFileFor(area, language)`, `hasTrack()`, `loadTrack()` resolve tracks. `TRACK_LANGUAGES` tells onboarding which languages have project lessons per area.
`services/gameCatalog.ts`	Injects mini-games into a general course at load time.	`addingGames(course, area)` inserts a loop-game lesson (id `p2g1`) as the third item of Phase 2 and appends a one-lesson "Stage" phase (id `p4`, lesson `p4g1`). Only for Healthcare, Engineering and Business & finance.
`services/apiClient.ts`	Optional backend client, switched off.	`AppConfig { backendURL, useBackendLessons, useAIExplanations }`. `APIClient.fetchCourse(area)`: GET `/lessons?area=`, 8 s timeout, null on failure. `APIClient.explain(body)`: POST `/explain`, 15 s timeout, throws on failure.
`services/purchases.ts`	Thin, never-throwing wrapper over RevenueCat.	`RevenueCatKeys` (test-store key, placeholder Apple key, entitlement id `premium`). `PurchasesService.configure, isPremium(info), customerInfo, currentOffering, purchase(pkg), restore, onUpdate(listener)`.
`storage/progressStorage.ts`	Load/save `UserProgress`.	`ProgressStorage.load()` (defensive field-by-field decode, validates robot forms), `save()`.
`storage/trackStorage.ts`	Load/save `TrackProgress`.	`TrackStorage.load(), save()`.

8.4 `src/hooks` and `src/navigation`

File	Purpose	Notes
`hooks/useAppStore.tsx`	App-wide store (section 7.1).	`AppStoreProvider`, `useAppStore()`, type `PaywallTier`. Two effects on mount: hydrate progress and course; configure RevenueCat and subscribe to entitlement updates.
`hooks/useTrackStore_updated.tsx`	Track store (section 7.2).	`TrackStoreProvider`, `useTrackStore()`, `DAILY_GOAL_MINUTES`.

`hooks/useReduceMotion.ts`	Accessibility setting.	Returns the OS Reduce Motion flag and updates live. Every animated component checks it.
`navigation/RootNavigator.tsx`	The gate and the root stack (section 6).	Exports `RootStackParamList`. Sets the navigation theme background to cream. Renders `PaywallOverlay` above the navigator.
`navigation/MainTabs_updated.tsx`	Bottom tabs.	Chooses `TrackPathScreen` or `LearnScreen` for the Learn tab with `hasTrack()`. Ionicons: book, construct, person-circle.

8.5 `src/screens`

File	Purpose	Important pieces
`SplashScreen.tsx`	Logo over a full-screen picture.	No logic; timing is in `RootNavigator`.
`OnboardingScreen_updated.tsx`	Two-step profile set-up.	Local components `AgeSlider` (PanResponder, accessible "adjustable" role), `Choice`. `goToLanguages()` preselects a valid language; the final button calls `store.finishOnboarding()`. Prefills from saved progress, so it doubles as "Edit my answers".
`LearnScreen.tsx`	General-course home path.	`rows` (banners and lessons, reversed), `lessonRow()` (zig-zag offsets `[0, 55, 0, -55]`, states done / current / locked), `banner()` (progress pill or Premium pill), `scrollToCurrent()`, `useFocusEffect` that opens Evolution, `openLesson()` that routes to `Lesson` or `GameLesson`.
`LessonScreen.tsx`	General-course lesson player.	Constants `BOLTS_PER_CORRECT` = 5, `BOLTS_FOR_FINISHING` = 10. `check()` grades by kind (build compares joined code and exact piece order; match compares each row's partner). `next()` advances the queue. `finish()` calls `completeLesson`. `questionBody()` renders the five kinds; `feedback()` is the bottom sheet; `explainMore()` calls the AI endpoint when enabled. Also exports `LessonTopBar` and `SimpleLessonComplete`, reused by the game screen.
`GameLessonScreen.tsx`	Host for mini-games.	Looks up `lesson.game` in `LOOP_CONFIGS` or `STAGES`, passes `onProgress, onBolts, onDone`, guards against double-awarding the finishing bolts, then completes the lesson like any other.
`games/LoopGame.tsx`	"Guess the loop count" game.	`LOOP_CONFIGS` (plant, wheel, fund), `steps(n)` builds the timed playback, `wrongText()` gives targeted feedback (zero, undershoot, the classic "forgot the starting value" trap, overshoot), SVG art components `PlantArt, WheelArt, FundArt`.
`games/StageGame.tsx`	Five-part "lab" game.	`build(spec)` expands a compact `StageSpec` into five parts; `STAGES` (gc, duty, conversion); `runSteps()` scripts the final animated run with captions and console output.
`EvolutionScreen.tsx`	Robot evolution reveal.	900 ms pause, then a spring cross-fade from the blob to the new form. Swipe-to-dismiss is disabled until the reveal.

`WorkshopScreen.tsx`	Robot wardrobe.	Grid of six forms (tap an owned one to `equip`) and three parts. `PARTS` is a hard-coded constant; parts have no behaviour yet.
`ProfileScreen.tsx`	Stats, badges, settings.	Streak hero with `WeekStrip`; lessons / concepts / bolts row; nine badges computed inline from both stores; read-only settings rows; buttons: Edit my answers, Reset all progress (native Alert), Subscriptions (opens the paywall); name editor modal.
`track/TrackPathScreen_updated.tsx`	Track home path.	`rows`, `currentID` (next unfinished lesson, else the checkpoint, else the first lesson under 5 stars), `banner()`, `nodeRow()`, cycle-2 teaser band at the top.
`track/TrackHomeScreen_updated.tsx`	The phase page.	`ctaFor(phase)`, `phaseCard()`, `cycleTwoCard()`, `laterRow()` for locked phases, a "phases done" summary card. Opens `TrackFlow` with `{ phase, lessonId }`.
`track/TrackFlowScreen_updated.tsx`	State machine for a lesson or checkpoint run (figure 4).	The `Step` union; `lessonStep()` picks V2 or the older player with `v2Lesson(id)`; `go()` bumps `runID` so replays remount the player.
`track/TrackPlayer.tsx`	Older lesson player, and the only checkpoint player.	`lesson === null` means checkpoint mode. `isRight` grades fill / predict / bug / build / order (order accepts any sequence listed in `accept`). `lessonBody` is the light layout with `ProjectCard`; `checkpointBody` has five navy layouts chosen by `q.style`. `advance()` records the run and credits bolts.
`track/TrackResults.tsx`	Four result and intro screens.	`TrackLessonComplete` (count-up total, stars, +30 tag, streak), `ProjectBuilt` (finished project card and code), `CheckpointIntro`, `CheckpointResult` (unlock card, or a list of what still needs 5 stars).

8.6 `src/components`

File	Purpose	Key exports
`ui.tsx`	Base building blocks used on every screen.	`Card` (fill, navy outline, optional hard drop shadow), `ChunkyButton` (the main button, presses down 4 pt), `BundleImage` and `imageSource(name)` (looks up `name + "_updated"` first, then `name`), `FullScreenPicture`, `DottedBackground`, `TokenChip`, `CodeBlock`, `FlowWrap`.
`trackKit.tsx`	Shared visuals for the track, also reused by general screens.	Icons (`BoltIcon, StarIcon, LockIcon, CheckIcon, CrossIcon, FlameIcon`); `useCountUp`, `useSlideUp`; `BoltPill` (animated bolt counter); `SegmentBar`, `StarSlots` (progress); `RobotFace`, `moodPlan(index)` (rotating expressions), `CoachRow`, `PromptBubble` (question with hint bulb); `colorR()`, `RCodeLine` (R syntax colouring, blank filling); `PhasePicture` (helix, quality, plate, growth, report, batch), `DNAStrip`; `ProjectCard`; `FeedbackSheet`; `QuitSheet`; `StarsRow`; `StatChipsRow` (streak / bolts / minutes chips with pop-up bubbles); `WeekStrip`.

`lessonKitV2_updated.tsx`	The complete new-style lesson: data types, registry, three screens.	Types `V2Pic, V2Question, V2Lesson`. Registry: both V2 JSON files are indexed by lesson id at import; `v2Lesson(id)` returns one or null. Screens: `LessonIntroV2`, `LessonPlayerV2` (grading, first-try tracking, science and code note panels, calls `track.record`), `LessonRecapV2`. Internal pieces: entrance animations (`Enter`), `Ring`, `QSlot`, `Carry`, `QuestionBar`, built-in pictures (`Strip, Readout, Rows, Bars, Tubes, PhScale, Machine, Message`), `CodeLine`, `Answers`, `Paper` (bug-hunt view), `Sheet`, `Notebook`. `colorPy()` colours Python.
`picturesV2.tsx`	Animated pictures for Engineering and Healthcare lessons.	`PictureV2({ p, state })` switches on `p.type`: belt (fruit conveyor), circuit, battery, drip, gauge, clock24, wristband, syringe, monitor (ECG), bpbars, curve. `PicState` is `'idle' | 'go' | 'nope'`: idle loops while thinking, "go" acts out the right answer, "nope" shakes. Helpers `useGo, useLoop, Frame, Tag, arg`.
`gameKit.tsx`	Mini-game toolkit.	`CodeToken, CodeLine, tk()` (tokenised code); `KaraokeLine`, `CodePanel` (highlights the running token, shrinks font to fit); `RunConsole`; `GameFeedbackSheet`; `FittedArt`; `RunStep` and `useStepPlayer()` (plays a list of timed steps, cancels on unmount).
`PaywallOverlay.tsx`	The subscription card.	`TIER` (title, price, robot, colours, feature bullets for premium and expert), `Confetti`, `PaywallOverlay`. Picks the RevenueCat package whose price string equals the tier price, else the first package. `startTrial()` calls `store.purchase`.
`SafeAreaView.tsx`	Drop-in replacement for the library's SafeAreaView.	Pads from `useSafeAreaInsets()` because the native view can report zero insets inside `fullScreenModal` screens. Use this one, not the library's.

9 Content formats

All content is in `assets/content`. There are three shapes.

9.1 General course: `lessons*.json` → `Course`

```
Course { version, area, phases: Phase[] }
└ Phase { id, number, title, subtitle, unlocksForm?: "builder", lessons: Lesson[] }
   └ Lesson { id, title, questions: Question[], game?: "loop_fund" | "stage_conversion" | ... }
      └ Question { id, kind, prompt, hint?, code?, picture?, options[], answer[], right?,
explanation }
```

kind	What the learner does	How `options` and `answer` are used
choice	Pick one chip. If `code` contains `___` the pick is shown filled into the blank.	`answer[0]` is the correct option text.
judge	Two big buttons (for example True / False).	`answer[0]` is the correct option.
bug	Tap the line that contains the mistake.	`options` are the code lines (leading spaces become indentation); `answer[0]` is the faulty line.
build	Tap pieces into a tray to build a line.	`options` are the pieces (may include distractors); `answer` is the exact ordered list of pieces.
match	Pair each line with its meaning.	`options` are the left rows; `answer[i]` is the correct partner for row *i*; `right` (optional) is the shuffled list of partner chips, defaulting to `answer`.

```
{ "id": "p1l1q4", "kind": "bug", "prompt": "Tap the line with the mistake.",
  "options": ["print(\"Order\")", "print(\"Ship\")", "print(Open!)"],
  "answer":  ["print(Open!)"],
  "explanation": "Words need quote marks. Fix: print(\"Open!\")" }
```

Some questions carry extra keys such as `fix` and `verify`. They look like authoring-tool metadata; the app ignores them.

9.2 Track structure: `track_*.json` → `Track`

```
Track { id, title, language, phases: TrackPhase[], cycle2: { area, project, picture, titles[],
later[][] } }
└ TrackPhase { number, area, project, picture, skill,
               parts: [label, value][]   // the four result rows on the project card
               code: string[]            // the finished script shown on "Project built"
               lessons: TrackLesson[], checkpoint: TrackCheckpoint }
   ├ TrackLesson { id, part (1 to 4), title, questions: TrackQuestion[5] }
   └ TrackCheckpoint { id, name, filename, problem, unlock: [tag, title, text], questions:
TrackQuestion[5] }
```

`TrackQuestion` is one wide type with optional fields per kind:

kind	Learner action	Fields used
fill	Choose what goes in the `___` blank.	`code[]`, `opts[]`, `correct` (id), `fb` (per-wrong-option feedback), `answerLine`, `screen.goal`
predict	Choose what the code prints.	`code[]`, `opts[]`, `correct`, `fb`, `screen.done / idle / idleLabel`
bug	Tap the line that causes the error.	`lines[]`, `correct`, `fb`, `screen.err` (the error shown) and `screen.done` (output after the fix)
build	Tap pieces to build the next line.	`code[]`, `opts[]`, `correct` (array of ids), `wrongBuild`; checkpoints add `report, reportTitle, slot`
order	Checkpoint only. Reorder steps with up/down arrows.	`steps` or `lines`, `start` (initial order), `accept` (every acceptable order), `wrong`, `goal`, `task`, `style` ("plan" or "code")

Every question also has `prompt, hint, ok` (the message when right) and optionally `note` (a dashed "field note" under the success message). Checkpoint questions set `style`, which selects the navy layout.

9.3 New-style lesson bodies: `track_bio_r_lessons_v2.json`, `track_lessons_v2_more.json` → `V2Lesson`

These are keyed by the *same lesson id* as the track file. If `v2Lesson(id)` finds an entry, the track file's own questions for that lesson are not played; they only supply the title, part number and question count used for the star threshold.

```
V2Lesson { id, domain?: "eng" | "health", lang?: "py" | "r", phase, part, title, project,
           person, intro,              // who is asking, and their message on the goal screen
           start,                      // label of the start button
           notebook: [label, value][], // "lab notebook" rows; filled in on the recap
           file, script: string[],     // the script the learner builds line by line
           doneTitle, doneSub, questions: V2Question[5] }

V2Question { id, kind: predict | fill | bug | build,
             word, color, okTitle,     // big heading ("Predict"), its colour, success title
             ask, hint, ok, fb, wrongBuild?,
             code[], start,            // lines shown and the line number they start at
             opts? | lines?, correct,
             pics: V2Pic[],            // the picture(s) above the code
             eq, goal, earlier, carry, // equation strip, goal chips, earlier-lines note, carried
value
             science: { title, text }, // "Tap for the biology / engineering / clinical context"
             codeNote: { title, text },// "What it does"
             bridge, entry, label, after? }
```

Pictures. Types drawn inside `lessonKitV2`: strip (DNA letters), readout, rows, bars, tubes, ph, machine, msg. Types drawn by `picturesV2` take positional `args` and named `kw`: belt, circuit, battery, drip, gauge, clock24, wristband, syringe, monitor, bpbars, curve. Example from the fruit-sorter lesson:

```
"pics": [{ "type": "belt", "args": [["spoilt_orange", "bin", "45%"], ["clean_apple", "floor",
null]]], "kw": {} }]
```

> **Question-id mismatch in the Biology track.** The Biology V2 questions are named `bio_p1_l1_v2_q1` while the track file's are `bio_p1_l1_q1`. Payouts use the V2 ids, which is fine, but Profile's "Bug hunter" badge counts paid ids against bug questions from the track file, so bug questions in Biology phases 1 to 3 never count toward it. The Engineering and Healthcare files use identical ids in both places.

10 Lesson plans

10.1 General course in Python (three phases, 12 lessons, 5 questions each)

Every study-area file follows the same skeleton with the same lesson ids, concepts and question mix; only the story changes. That is why progress carries over when a learner switches area. Phase 1 completion unlocks the Builder Bot. The last three columns are bundled but not currently reachable from the UI (section 4).

Lesson id	What it teaches	Question mix	General	Business & finance	Physics & maths	Arts & humanities	Life sciences	Engineering	Computing
Phase 1: Say hello · finishing it unlocks the Builder Bot form									
`p1l1`	print() and text in quotes; spot a missing-quotes bug	3 choice, 1 build, 1 bug	Screen	Shop screen	Lab screen	Theatre board	Garden sign	Site board	Terminal
`p1l2`	Numbers and + - *; "12 + 8" in quotes is text, not maths	3 choice, 1 judge, 1 build	Counting things	Counting orders	Counting readings	Counting chapters	Counting leaves	Counting beams	Counting files
`p1l3`	Variables: assign with =, read back, reassign	3 choice, 1 judge, 1 match	Labelled boxes	Labelled shelves	Labelled meters	Labelled notebooks	Labelled pots	Labelled crates	Labelled boxes
`p1l4`	Updating a variable from itself (x = x - 25)	2 choice, 1 build, 1 bug, 1 match	Update day	Restock day	Speeding up	Writing day	Watering day	Crane lifts	Loading day
Phase 2: Loops: repeat a step									
`p2l1`	for ... in range(n); which lines sit inside the loop	3 choice, 1 build, 1 judge	Rounds pass	Weeks pass	Ticks pass	Chapters pass	Days pass	Shifts pass	Ticks pass
`p2l2`	Changing a variable inside a loop (running totals)	4 choice, 1 judge	Watch it change	Watch sales grow	Watch it move	Watch it grow	Watch it grow	Watch it rise	Watch it count
`p2l3`	The loop variable; range(n) vs range(a, b)	3 choice, 1 match, 1 build	Counting rounds	Counting weeks	Counting seconds	Counting scenes	Counting days	Counting hours	Counting steps
`p2l4`	Multi-line loop bodies; picking the right loop; bug hunt	4 choice, 1 bug	Routine	Shop routine	Lab routine	Rehearsal routine	Garden routine	Site routine	Server routine
Phase 3: Decisions: if / else									
`p3l1`	if with ==; what runs when the test is False	3 choice, 1 judge, 1 build	On or off	Full or empty	On or off	Open or closed	Wet or dry	Wet or set	Up or down
`p3l2`	Comparisons > < >= and else	4 choice, 1 match	More or less	Busy or quiet	Fast or slow	Loud or soft	Sunny or not	Safe or not	Fast or slow
`p3l3`	if inside a for loop	4 choice, 1 bug	Round check	Weekly check	Sensor check	Nightly check	Daily check	Site check	Health check
`p3l4`	Mixed review of phases 1 to 3	3 choice, 1 judge, 1 build	Review	Shop review	Lab review	Show review	Garden review	Site review	Code review

Themes at a glance: General (screens and rounds), Business (a shop: orders, stock, sales), Physics & maths (lab meters and readings), Arts & humanities (theatre and writing), Life sciences (a garden), Engineering (a building site), Computing (a server). `lessons.json` is a copy of the garden course used as the fallback.

10.2 Language starter courses (Phase 1 only)

Language / file	Phase 1	Theme	Lessons (5 questions each)	Teaches	Id pattern
R `lessons_r.json`	Say hello: Show messages and store numbers	Hospital ward	Ward sign, Reading numbers, Patient labels, Dosing	print(), numbers vs text, assignment with <-, updating a variable	`r_p1l1 ...`
SQL `lessons_sql.json`	Ask the data: Read a table with SELECT	Cafe menu table	Read the menu, Pick the rows, Put in order, Count and add	SELECT ... FROM, WHERE, ORDER BY, counting with COUNT(*)	`sql_p1l1 ...`

JavaScript `lessons_javascript.json`	Say hello: Show messages and store numbers	Video game	Game start, Coins and text, Named boxes, Level up	console.log(), numbers vs text, let variables, updating a variable	`javascript_p1l1 ...`
Java `lessons_java.json`	Say hello: Show messages and store numbers	Robot workshop	Workshop screen, Numbers and text, Typed boxes, Robot maths	System.out.println(), numbers vs text, typed variables (int), arithmetic	`java_p1l1 ...`
MATLAB `lessons_matlab.json`	Say hello: Show messages and store numbers	Electronics bench	Bench display, Numbers and text, Bench readings, Distance	disp(), numbers vs text, variables with =, simple formulas	`matlab_p1l1 ...`

Each unlocks the Builder Bot on completion, like the Python Phase 1. Onboarding tells the learner "starts with Phase 1. More phases are coming soon."

10.3 Project tracks

A phase is one project built across four parts, then a checkpoint. "New style (V2)" lessons have a goal screen, animated picture, science note and recap; "Older TrackPlayer" lessons show the project card and code only. In V2 lessons the question order is nearly always Predict, Fill, Predict, Debug, Build.

Biology research in R `(track_bio_r_updated.json, track id bio_r)`

Id	Lesson	Scenario shown on the goal screen	Question order	Player
Phase 1 · Bioinformatics · Project: DNA sequence report · Skill: Basics · finished script: 7 lines				
`bio_p1_l1`	Part 1: **Print the sequence** `sample_report.R`	*Sequencing facility:* Your sample S07 is ready: ATGCGC. Please check it before we run the full batch.	Predict, Fill, Predict, Debug, Fill	New style (V2)
`bio_p1_l2`	Part 2: **GC content** `gc_check.R`	*Lab partner:* My notebook says this sample is 35.3% GC. Can you check it in R?	Predict, Fill, Predict, Debug, Build	New style (V2)
`bio_p1_l3`	Part 3: **Count the AT bases** `codon_count.R`	*Classmate on the forum:* Can someone check this 10-base read with me? I keep getting odd numbers.	Predict, Fill, Predict, Debug, Build	New style (V2)
`bio_p1_l4`	Part 4: **Both strands** `strands.R`	*Poster reviewer:* Your poster shows both DNA strands. I will check the pairing before the session.	Predict, Fill, Predict, Debug, Fill	New style (V2)
`bio_p1_cp`	**Checkpoint: Primer check** `primer_tm.R` Before PCR, labs estimate a primer's melting temperature, Tm. For short primers the Wallace rule gives Tm = 2(A+T) + 4(G+C).			
Phase 2 · Genomic processing · Project: Read quality checker · Skill: Functions · finished script: 7 lines				
`bio_p2_l1`	Part 1: **Store the scores** `read_scores.R`	*Sequencer manual:* Quality scores are exported one per base. Position 0 is the first base.	Predict, Fill, Predict, Debug, Build	New style (V2)
`bio_p2_l2`	Part 2: **Average with mean()** `read_summary.R`	*Lab partner:* My spreadsheet average for this read is 23.2. Does R agree?	Predict, Fill, Predict, Fill, Debug	New style (V2)
`bio_p2_l3`	Part 3: **Write avg_quality()** `quality_tools.R`	*Your TA:* Please wrap the quality check in a function, so you can reuse it on every read.	Predict, Fill, Predict, Debug, Build	New style (V2)
`bio_p2_l4`	Part 4: **Check a second read** `compare_reads.R`	*Collaborator:* Here is a second read from the same sample. My script says both reads pass.	Predict, Fill, Predict, Build, Debug	New style (V2)
`bio_p2_cp`	**Checkpoint: Protein mass** `protein_mass.R` Biochemists estimate a protein's mass from its length: each amino acid adds about 110 daltons on average.			

Phase 3 · Automation and lab efficiency · Project: Plate dilution planner · Skill: Decisions · finished script: 7 lines				
bio_p3_l1	Part 1: **Too big for a well?** well_check.R	*Lab manual:* A well on this plate holds at most 200 µL. Check every volume before you pipette.	Predict, Fill, Predict, Debug, Build	New style (V2)
bio_p3_l2	Part 2: **Big or small sample** sample_plan.R	*Clinic sample sheet:* Three samples: 3 million, 0.5 million and 2.5 million cells per mL. Dilute anything at 1 million or more.	Predict, Fill, Predict, Debug, Build	New style (V2)
bio_p3_l3	Part 3: **Fix the else line** ph_check.R	*Group chat:* Anyone free to look at my pH check? My if/else won't run at all.	Predict, Fill, Predict, Debug, Build	New style (V2)
bio_p3_l4	Part 4: **Buffer to add** dilution_recipe.R	*Reagent datasheet:* Stock concentration 10x. Make 50 µL at 2x. All volumes in µL.	Predict, Fill, Predict, Debug, Fill	New style (V2)
bio_p3_cp	**Checkpoint: Disk test** disk_test.R Labs measure the clear zone around an antibiotic disk. For a 5 µg ciprofloxacin disk, proposed cut-offs are 21 mm or more susceptible, 16 to 20 intermediate, 15 or less resistant.			
Phase 4 · Modelling and simulation · Project: Bacterial growth simulator · Skill: Loops · finished script: 7 lines				
bio_p4_l1	Part 1: **Loop over hours**	No goal screen; plays straight into the questions.	predict, fill, predict, bug, fill	Older TrackPlayer
bio_p4_l2	Part 2: **Double the cells**	No goal screen; plays straight into the questions.	predict, fill, predict, build, predict	Older TrackPlayer
bio_p4_l3	Part 3: **Count the hours**	No goal screen; plays straight into the questions.	predict, fill, predict, bug, bug	Older TrackPlayer
bio_p4_l4	Part 4: **Last count**	No goal screen; plays straight into the questions.	predict, fill, predict, predict, build	Older TrackPlayer
bio_p4_cp	**Checkpoint: Drug clearance** clearance.R After a dose, the level of many drugs halves every half-life. Here, 80 mg/L halves every 6 hours.			
Phase 5 · Reproducibility · Project: Reproducible analysis report · Skill: Lists and data · finished script: 5 lines				
bio_p5_l1	Part 1: **Set the seed**	No goal screen; plays straight into the questions.	fill, predict, fill, bug, fill	Older TrackPlayer
bio_p5_l2	Part 2: **Tally species**	No goal screen; plays straight into the questions.	predict, fill, predict, predict, predict	Older TrackPlayer
bio_p5_l3	Part 3: **Seed goes first**	No goal screen; plays straight into the questions.	fill, predict, predict, bug, bug	Older TrackPlayer
bio_p5_l4	Part 4: **Report list**	No goal screen; plays straight into the questions.	fill, predict, predict, bug, build	Older TrackPlayer
bio_p5_cp	**Checkpoint: Blood bank sample** blood_bank.R A blood bank tallies the blood groups of a random sample of donors. The script must give the same sample every time it runs.			
cycle2	**Teaser only, not playable:** Bioinformatics, round 2 · Batch GC report · parts: Loop over samples, Flag unusual GC, Wrap it in a function, Sort the results Later rows: Genomic processing, round 2: Genetic fingerprint picker (Sorting); Automation, round 2: Batch plate planner (Lists); Modelling, round 2: Biodiversity simulation (Random numbers)			

Engineering in Python (track_eng_py.json, track id eng_py)

Id	Lesson	Scenario shown on the goal screen	Question order	Player
Phase 1 · Engineering · Project: Circuit calculator · Skill: Values, maths and printing · finished script: 11 lines				

EngP1L1	Part 1: **Label the circuit** `circuit.py`	*Lab technician:* Before anyone powers a board, it needs a label: its name and its supply voltage.	Predict, Fill, Predict, Debug, Fill	New style (V2)
EngP1L2	Part 2: **Ohm's law** `ohms_law.py`	*Lab partner:* Our 12 V supply drives a 4 Ω resistor. Can you check the current and power before we switch on?	Predict, Fill, Predict, Debug, Build	New style (V2)
EngP1L3	Part 3: **Battery life** `battery.py`	*Robotics club member:* Our robot runs on a 2000 mAh battery and draws 300 mA. How long will it last between charges?	Predict, Fill, Predict, Debug, Build	New style (V2)
EngP1L4	Part 4: **Power budget** `power_budget.py`	*Senior engineer:* Before we order a supply, add up what every part draws and check we keep 20% headroom.	Predict, Fill, Predict, Debug, Build	New style (V2)
eng_py_cp1	*Checkpoint not written yet (empty questions array, name "Coming soon")*			
Phase 3 · Engineering · Project: Fruit sorting conveyor · Skill: Decisions: if, elif, else · finished script: 19 lines				
EngP3L1	Part 1: **Sort the fruit** `sorter.py`	*Internship mentor:* The colour sensor reads how much of each fruit is covered in dark spots. Over 30% is spoilt, and the pusher sends it to the bin.	Predict, Fill, Predict, Debug, Build	New style (V2)
eng_py_cp3	*Checkpoint not written yet (empty questions array, name "Coming soon")*			
cycle2	**Teaser only, not playable:** Engineering · More phases on the way			

Healthcare in Python `(track_health_py.json, track id health_py)`

Id	Lesson	Scenario shown on the goal screen	Question order	Player
Phase 1 · Healthcare · Project: Dose calculator · Skill: Values, maths and printing · finished script: 8 lines				
HpyP1L1	Part 1: **Dose by weight** `dose_calc.py`	*Ward pharmacist:* Training example: Drug X is dosed at 10 mg per kg of body weight, up to 500 mg. Check each dose in code before it reaches the ward.	Predict, Fill, Predict, Debug, Build	New style (V2)
HpyP1L2	Part 2: **Infusion rate** `infusion.py`	*ICU nurse:* Training example: 1000 mL of fluid over 8 hours, through a giving set that delivers 20 drops per mL. Work out the rates in code.	Predict, Fill, Predict, Debug, Build	New style (V2)
HpyP1L3	Part 3: **BMI check** `bmi.py`	*Dietitian:* Training example: a patient weighs 70 kg and is 175 cm tall. Work out their BMI, the way the clinic form does.	Predict, Fill, Predict, Debug, Build	New style (V2)
HpyP1L4	Part 4: **Next dose time** `dose_times.py`	*Ward pharmacist:* Training example: Drug X is given every 6 hours, 500 mg each time, up to 2000 mg a day. The last dose was at 22:00.	Predict, Fill, Predict, Debug, Build	New style (V2)
health_py_cp1	*Checkpoint not written yet (empty questions array, name "Coming soon")*			
Phase 3 · Healthcare · Project: Early warning alert · Skill: Decisions: if, elif, else · finished script: 19 lines				
HpyP3L1	Part 1: **Early warning alert** `news2_pulse.py`	*Night-shift nurse:* NEWS2 scores each vital sign. For pulse: 51 to 90 scores 0, 91 to 110 scores 1, 111 to 130 scores 2, 131 or more scores 3.	Predict, Fill, Predict, Debug, Build	New style (V2)
health_py_cp3	*Checkpoint not written yet (empty questions array, name "Coming soon")*			
cycle2	**Teaser only, not playable:** Healthcare · More phases on the way			

Healthcare in R `(track_health_r.json, track id health_r)`

Id	Lesson	Scenario shown on the goal screen	Question order	Player
Phase 1 · Healthcare · Project: Clinic blood pressure log · Skill: Values, vectors and printing · finished script: 5 lines				
HrP1L1	Part 1: **Blood pressure log** `bp_log.R`	*Clinic nurse:* This week's systolic readings for one patient: 128, 140, 135 and 151. The clinic flags any reading of 140 or above for a follow-up.	Predict, Fill, Predict, Debug, Build	New style (V2)

`HrP1L2`	Part 2: **Mean arterial pressure** `map.R`	*Clinic doctor:* For this reading of 120 over 80, I need the pulse pressure and the mean arterial pressure, MAP, for the notes.	Predict, Fill, Predict, Debug, Build	New style (V2)
`HrP1L3`	Part 3: **Patient ID labels** `id_labels.R`	*Medical records officer:* Training example: the clinic system exports IDs like s-0042 in lowercase. Wristbands need them in a standard ward format.	Predict, Fill, Predict, Debug, Build	New style (V2)
`HrP1L4`	Part 4: **Glucose log** `glucose.R`	*Diabetes educator:* Training example: four blood glucose readings this week, in mmol/L. The clinic in this example flags anything above 10 or below 4.	Predict, Fill, Predict, Debug, Build	New style (V2)
`health_r_cp1`	*Checkpoint not written yet (empty questions array, name "Coming soon")*			
Phase 4 · Healthcare · Project: Outbreak curve · Skill: Loops · finished script: 13 lines				
`HrP4L1`	Part 1: **Outbreak curve** `outbreak.R`	*Public health officer:* New cases reported each day this week: 3, 5, 9 and 14. I need the running total and the trend for this afternoon's briefing.	Predict, Fill, Predict, Debug, Build	New style (V2)
`health_r_cp4`	*Checkpoint not written yet (empty questions array, name "Coming soon")*			
`cycle2`	**Teaser only, not playable:** Healthcare · More phases on the way			

10.4 Biology checkpoints in detail

Each checkpoint sets a fresh problem that uses the phase's skill, and always runs the same five styles in order: plan the steps, order the real code, predict the output, find the bug, build the last line.

Checkpoint	Its 5 questions (style, kind, prompt)	Unlock card shown at 5/5
Phase 1 **Primer check** `primer_tm.R`	**plan** (order): Let's do some planning first! **code** (order): Order the real R code **predict** (predict): What does R print? **debug** (bug): Find the line that causes the error **build** (build): PCR anneals about 5 °C below Tm. Show that temperature	**Phase 2 unlocked**: Genomic processing. New skill: functions. You'll build a read quality checker.
Phase 2 **Protein mass** `protein_mass.R`	**plan** (order): Plan the function **code** (order): Order the function code **predict** (predict): What does R print? **debug** (bug): Which line causes the error? **build** (build): Estimate alpha-synuclein's mass (140 amino acids)	**Phase 3 unlocked**: Automation and lab efficiency. New skill: decisions. You'll build a plate dilution planner.
Phase 3 **Disk test** `disk_test.R`	**plan** (order): Plan the decision **code** (order): Order the decision code **predict** (predict): What does R print for a 16 mm zone? **debug** (bug): Which line makes R stop? **build** (build): Print Resistant for zones of 15 mm or less	**Phase 4 unlocked**: Modelling and simulation. New skill: loops. You'll build a bacterial growth simulator.
Phase 4 **Drug clearance** `clearance.R`	**plan** (order): Plan the loop **code** (order): Order the loop code **predict** (predict): What does R print? **debug** (bug): This loop never ends. Which line is wrong? **build** (build): Each half-life is 6 hours. Show the total time	**Phase 5 unlocked**: Reproducibility. New skill: lists and data. You'll build a reproducible analysis report.
Phase 5 **Blood bank sample** `blood_bank.R`	**plan** (order): Plan a repeatable script **code** (order): Order the script **predict** (predict): What does R print? **debug** (bug): The report prints NULL. Which line is wrong? **build** (build): Show how many donors are in the sample	**Cycle 2 unlocked**: Bioinformatics, round 2. You've been here before. Now build a batch GC report with everything you know.

10.5 The curriculum arc

Stage	General course	Biology track (the most complete)	Engineering / Healthcare tracks
1	Printing, numbers, variables	Basics: print, arithmetic, variables (DNA sequence report)	Values, maths and printing (Circuit calculator; Dose calculator; Clinic blood pressure log)

2	for loops and range()	Functions: vectors, mean(), writing a function (Read quality checker)	not written
3	if / else and comparisons	Decisions: if / else (Plate dilution planner)	Decisions: if, elif, else (Fruit sorting conveyor; Early warning alert), one sample lesson each
4	Stage mini-game: strings, functions, loops together (business only)	Loops (Bacterial growth simulator)	Loops (Outbreak curve), one sample lesson, Healthcare R only
5		Lists and data, reproducibility (Reproducible analysis report)	not written
Cycle 2		Teaser: the same five areas again with bigger projects	"More phases on the way"

11 Mini-games

Mini-games are general-course lessons with a `game` key and no questions. They are added at load time by `gameCatalog.addingGames()` rather than stored in the JSON. Both kinds teach Python and both are built on `gameKit`'s token-by-token "karaoke" code playback.

Loop games (`games/LoopGame.tsx`)

The learner sees three lines of code with one blank, `range(?)`, types a number from 0 to 20 and presses "Test!". The code then runs step by step while the art animates. The trap in every variant is the non-zero starting value.

Game id	Field	Task	Start, step, goal	Answer	Trap answer	Reachable?
`loop_plant`	Healthcare	Grow the plant to 10 cm	2, +1, 10	8	10	No (Healthcare + Python now gets a track)
`loop_wheel`	Engineering	Spin the wheel to 9 turns	3, +1, 9	6	9	No
`loop_fund`	Business & finance	Grow the fund to $4,000	1000, +500, 4000	6	8	Yes

Scoring: 5 bolts if right on the first test, plus 10 for finishing.

Stage labs (`games/StageGame.tsx`)

A five-part capstone that assembles one small program: a function that counts characters in a string and divides by the length, called in a loop over three items.

Part	Name	Kind	What happens
1	String method	choice	What does `"ATGCGT".count("G")` give?
2	Build the function	order	Tap the three lines (`def`, count, `return`) into the right order.
3	Call it	choice	What does the function return for a given input? Distractors are the count, the length and another item's ratio.
4	Loop it	choice	Which name goes in `for x in ___:`? (the list, not the function, a string or `len()`)
5	Run it	run	Watch the whole program execute over three items with captions and console output.

Game id	Field	Function	Data	Reachable?
`stage_gc`	Healthcare	`gc_content(sequence)`: share of G and C	ATGCGT, GGCAAT, TTATGC	No
`stage_duty`	Engineering	`duty_cycle(signal)`: share of 1s	1100, 1110, 1000	No
`stage_conversion`	Business & finance	`conversion_rate(log)`: share of S (sales)	SNSNN, SSSNN, NNNNS	Yes

Scoring: 5 bolts per part completed without a miss, plus 10 for finishing (maximum 35).

12 Payments and the backend hooks

12.1 Paywall and RevenueCat

Where it opens	Tapping a locked phase banner or a lesson in a not-yet-reached phase on `LearnScreen`; the "Subscriptions" button on Profile. Both pass `'premium'`. Nothing ever opens the `'expert'` tier, and track screens never open the paywall.
What it shows	`PaywallOverlay`: blurred backdrop, confetti, tier card, price from the RevenueCat package (fallback to the hard-coded $9.90 / $19.90), "Start FREE 7-day Trial".
Purchase path	`startTrial()` → `store.purchase(pkg)` → `PurchasesService.purchase` → `isPremium` set from the `premium` entitlement. The overlay closes on success or shows "Purchase didn't go through".
Configuration	`services/purchases.ts`: `useTestStore: true` with a RevenueCat test-store key committed in source; `appleAPIKey` is a placeholder; there is no Google key.
Expo Go	The SDK runs in Preview API Mode: offerings and entitlement calls work, purchases are mocked. Real purchases need a development build (`expo-dev-client` + EAS).

What is missing before this can ship.

- **Entitlement gating.** `isPremium` is stored but never read. Decide what Premium does (the banner says "skip with Premium"; the tier bullets promise level limits, feedback, streak restores, an AI chatbot) and implement it, or change the copy.
- **Restore purchases.** `restorePurchases()` exists in the store with no button. App Store review requires one.
- **Keys.** Move keys to build-time config, add the real Apple and Google keys, set `useTestStore` false for release.
- **Package selection.** Matching a package by its formatted price string breaks for any non-USD storefront; select by package or product identifier.

12.2 Backend hooks (all off)

`AppConfig` in `services/apiClient.ts` points at `http://localhost:8000` with both feature flags false. No server code is in this folder. The client defines this contract:

Endpoint	Used when	Contract
`GET /lessons?area=<studyArea>`	`useBackendLessons` is true and the language is Python; fired in the background after hydration.	Returns a `Course`. 8-second timeout; any failure is ignored and the bundled course stays. A remote course replaces the bundled one in memory only, and does not go through `addingGames`.
`POST /explain`	`useAIExplanations` is true and the learner taps "Explain more" after a wrong answer in `LessonScreen`.	Body `{ questionPrompt, code, learnerAnswer, correctAnswer, age, studyArea }`; returns `{ explanation }`. 15-second timeout; on failure the app shows a fallback sentence. A request counter discards stale replies.

13 Design system and accessibility

Visual language

- **Look:** cream backgrounds, thick 2.5 pt navy outlines, flat fills, hard offset shadows (a navy block 3 to 4 pt below the element, not a blur).
- **Core colours** (`C` in `theme.ts`): cream `#FBF4E4`, card `#FFFDF6`, navy `#1B2472`, lime `#B5E04A` (correct, primary button), sky `#59B4FF` (current), orange `#FF8A3D` (wrong, retry), sun `#FFC93C` (stars, checkpoints).
- **Code surfaces:** dark editor `#0B0F3D` / screen `#171E63` with lime strings, orange numbers and keywords, blue function names.
- **Fonts:** Baloo 2 ExtraBold for headings, Atkinson Hyperlegible for body, Menlo or the platform monospace for code, Horizon for the MOAT wordmark and paywall price.
- **Design source:** comments refer to a "design canvas" with named pages; timings and sizes are copied from it. Ask for access.

Robot art

- Six forms: blob, builder, rover, gardener, orb, cat. Only blob and builder are obtainable (`isInVersionOne`).
- Seven moods per form plus a base image: neutral, excited, surprised, wink, closed, hearts, confused. File name: `<form>_<mood>_updated.png`; access with `moodImage(form, mood)` and `BundleImage`.
- `moodPlan(index)` rotates expressions across questions so the robot does not repeat itself.

Accessibility

- `accessibilityRole`, labels, selected/disabled/expanded states and live regions throughout.
- 44 pt minimum touch targets.
- `useReduceMotion()` disables or shortens every animation.
- Decorative images are hidden from screen readers.
- Haptics on answer check (success / error).

14 Known issues and gaps

Everything below was confirmed by reading the code or running a check, not inferred from the README. Tags: BUG wrong behaviour, GAP unfinished feature, CLEANUP hygiene, NOTE behaviour to be aware of.

Tag	Issue	Detail and where
CLEANUP	Stale root files break the type-check	`LearnScreen.tsx`, `TrackPathScreen.tsx`, `ui.tsx` in the project root. Deleting them brings `npm run typecheck` from 19 errors to 0.
CLEANUP	No version control, tests, lint or CI	Not a git repository. No test runner, no ESLint config (despite eslint-disable comments), no `eas.json`, no app icons or bundle identifiers in `app.json`.
CLEANUP	`_updated` suffixes and outdated README	Eight source files and most images carry the suffix. The README's structure and parity tables use the old names and its "Not verified" section no longer applies.
GAP	Premium unlocks nothing	`isPremium` is never read by a screen; the Expert tier is never opened; no restore button. Section 12.1.
GAP	Engineering and Healthcare tracks are thin	Phase 1 plus one later sample lesson each; all checkpoints are empty placeholders. A learner who 5-stars everything lands on the "More phases on the way" card.
BUG	Track UI hard-codes Biology's shape	"Part N of 4" (`TrackPlayer`, `ProjectCard`, `TrackLessonComplete`, `LessonIntroV2`), "N of 5" and "5 stars" (`TrackPathScreen`, `CheckpointResult`), the code-card label "R" and "· R" (`TrackPlayer`, `ProjectBuilt`), "Phase N+1 opens at 5 stars" (`TrackHomeScreen`; Engineering goes from phase 1 to phase 3), cycle-2 badge numbers 6 and 7+ (`TrackHomeScreen`). Wrong for tracks with a different number of parts, another language, or non-consecutive phase numbers.
BUG	No phase picture for Engineering and Healthcare	Their track files have `picture: ""`, and `PhasePicture` only knows the six Biology kinds, so the phase card shows an empty space.
BUG	Bolt copy does not match the rules	`StatChipsRow` says "Every right answer earns 10 bolts" (general lessons pay 5) and promises "20 bonus bolts" for hitting the daily minutes goal; no code awards that bonus.
BUG	"Bug hunter" badge under-counts in Biology	V2 question ids differ from the track file's ids for Biology phases 1 to 3. Section 9.3.
BUG	Health and engineering mini-games are orphaned	`loop_plant, loop_wheel, stage_gc, stage_duty` cannot be reached now that those areas have Python tracks. Either surface them inside the tracks or remove them.
BUG	Track learners cannot evolve the robot	`creditTrack` never unlocks forms, so the "Hatched" badge and the Builder Bot ("Finish Phase 1" in the Workshop) are unobtainable on any track.
BUG	Evolution copy assumes a blob	`EvolutionScreen` always animates from the blob image and says "Your blob evolved!". Fine while Builder Bot is the only evolution.
NOTE	General lessons can be replayed for bolts	`completeLesson` adds bolts every time; completed lessons stay tappable. The track has a paid-once rule; the general course does not.
NOTE	Daily-goal data is half-wired	`dailyGoalLessons` and `lessonsToday` are stored and computed but never displayed. The visible goal is `DAILY_GOAL_MINUTES` = 10, and minutes are recorded only in track lessons, so general-course learners always see 0 minutes.
NOTE	Some badges assume the Biology track	"Loop master" counts finished lessons in the phase numbered 4; "concepts" counts phases whose first lesson is finished; "Perfect lesson" needs any item with 5 stars. General-course learners cannot earn several badges.
NOTE	Streak depends on the device clock	`DayKey` uses local time. Changing time zone or the clock can break or extend a streak. There is no server to arbitrate.
NOTE	Navigation during render	`TrackFlowScreen` calls `navigation.goBack()` in the render body when the phase is missing. Rare, but it should move into an effect.
NOTE	Secrets and config in source	RevenueCat test key and `localhost` backend URL are constants. Move to environment-based config before release.

CLEANUP	Dead code and dependencies	Unused: `@react-native-community/slider`; exports `HScroll` (trackKit) and `textStyles` (ui); content files `lessons_computing.json` and effectively `lessons.json`. `@types/react-native` 0.72 is listed although React Native has shipped its own types for several versions; it is worth removing and re-running the type-check.
NOTE	Large single-file components	`lessonKitV2_updated.tsx` (993 lines), `trackKit.tsx` (805), `picturesV2.tsx` (612). They work, but splitting by concern (types and registry, player, pictures) will make changes safer.

15 How-to recipes

Edit or add a general-course lesson

1. Edit the right `assets/content/lessons_<area>.json`. Follow the shape in section 9.1; keep five questions per lesson.
2. Give every new lesson and question a unique, permanent id. Never rename an existing id.
3. If the lesson should exist for all areas, add it to each area file with the same id so progress stays portable.

Add a new content file, study area or language

1. Add the JSON to `assets/content` and register it in `src/utils/content_updated.ts`.
2. Study area: add it to `STUDY_AREAS` in `utils/studyAreas.ts` and, if the file name is not the lower-cased area name, to the map in `contentLoader.fileName()`.
3. Language: add it to `languageFileName()` in the content loader and to `LANGUAGES` in `OnboardingScreen_updated.tsx`.

Add a project track, or a phase to one

1. Write `track_<area>_<lang>.json` (section 9.2) and register it in `content_updated.ts`.
2. Route to it in `trackFileFor()` and list the language in `TRACK_LANGUAGES` so onboarding promotes it.
3. For each new-style lesson, add a `V2Lesson` with the same id to `track_lessons_v2_more.json`. Use the same question ids in both files.
4. Give the checkpoint five questions, one per style, or leave `questions: []` to hide it.
5. Check the hard-coded strings listed in section 14 if your phase does not have exactly four parts of five questions.

Add a picture type for new-style lessons

1. Add the type to the `V2Pic` union in `lessonKitV2_updated.tsx`.
2. Implement a component in `picturesV2.tsx` taking `{ p: Args, state: PicState }`; wrap it in `Frame`, use `useLoop` for the idle motion and `useGo` for the "right answer" motion, and respect Reduce Motion.
3. Add a `case` to the `PictureV2` switch. Reference it from JSON with `{ "type": "...", "args": [...], "kw": {...} }`.

Add an image or robot expression

1. Drop the PNG in `assets/images` and add a line to `utils/images.ts` (it says "generated", but no generator script is in the folder, so edit by hand).
2. Render with `<BundleImage name="..."/>`; it tries `name + "_updated"` first.

Add a mini-game variant

1. Loop game: add an entry to `LOOP_CONFIGS` and an art component in `LoopGame.tsx`. Stage lab: add a `build({...})` entry to `STAGES`.
2. Map the study area to it in `gameCatalog.ts`.

Change scoring or unlock rules

Rule	Where
General bolts per answer / per lesson	`BOLTS_PER_CORRECT`, `BOLTS_FOR_FINISHING` in `LessonScreen.tsx`
Track bolts, perfect bonus, stars	`record()` in `useTrackStore_updated.tsx`
Phase / lesson / checkpoint unlocking on a track	`phaseComplete, phaseUnlocked, lessonUnlocked, checkpointUnlocked` in the same file
General lesson unlocking, robot evolution	`isUnlocked`, `completeLesson` in `useAppStore.tsx`

Streak	`bumpDay` and `displayStreak` in `useAppStore.tsx`
Daily minutes goal	`DAILY_GOAL_MINUTES` in `useTrackStore_updated.tsx`
Badges	The `badges` array in `ProfileScreen.tsx`

Turn on the backend or real purchases

- Backend: set `AppConfig.backendURL` and the two flags in `services/apiClient.ts`, and build the two endpoints in section 12.2.
- Purchases: `npx expo install expo-dev-client`, create an EAS development build, set real keys and `useTestStore: false` in `services/purchases.ts`, create the `premium` entitlement and an offering in RevenueCat.

16 Suggested first week, and glossary

A sensible order of work

1. **Put it under version control.** `git init`, commit the folder as received so there is a baseline.
2. **Get to a clean type-check.** Delete the three stale root files; confirm `npm run typecheck` passes; remove `@types/react-native` and the unused slider package and confirm again.
3. **Play every path on a device.** Biology + R (full track through a checkpoint), Engineering + Python, Healthcare + Python and + R (new pictures), Business + Python (mini-games), General + SQL (language starter). Compare against the list in `TESTING_playwright-cli.md`.
4. **Ask for the missing context:** the design canvas, the original Swift project, the "to-do doc" mentioned in `lessonKitV2`, the tool that generates the content JSON and the image/content registries, and the RevenueCat project.
5. **Get the product decisions made:** what Premium and Expert actually gate; whether track learners should evolve the robot; whether the orphaned mini-games move into tracks; lessons-per-day versus minutes-per-day as the daily goal.
6. **Make the track UI data-driven.** Replace the hard-coded "of 4", "of 5", "R" and phase-number arithmetic with values from the track file. This unblocks every non-Biology track.
7. **Add a safety net.** Unit-test the pure rules first (`record`, `bumpDay`, `isUnlocked`, `phaseComplete`, `loadBundled`, `codeJoin`) and add a JSON schema check for content files so a bad lesson fails in CI rather than on a phone.
8. **Then content:** checkpoints for Engineering and Healthcare, the missing phases, and converting Biology phases 4 and 5 to the new style.

Glossary

Bolts	The points currency. One global total in `progress.bolts`.
Stars	Track only. The number of questions in a lesson or checkpoint answered right on the first try; the best score is kept.
Phase	A group of lessons. In the general course, a concept; in a track, one project plus a checkpoint.
Part	A track lesson's position within its phase (1 to 4).
Checkpoint	The five-question test at the end of a track phase, in navy "exam mode".
Cycle	One pass through a track's phases. Cycle 2 (the same areas with bigger projects) is a teaser only.
Track	The project-based curriculum for one study area and language.
New style / V2	Lessons played by `LessonPlayerV2`: goal screen, animated picture, notes, recap.
Form, mood	The robot's body (blob, builder, ...) and its facial expression.
Stage	The five-part lab mini-game appended as a final general-course phase.
Canvas	The external design file the UI was built from. Not in this folder.
Paid	The list of track question ids that have already paid bolts, so replays do not pay twice.